In [1]:
```
import pandas as pd
```

In [2]:
```
sbux = pd.read_csv("BUS303_Signature_Term_Project_Starbucks_Beverage_Menu.csv")
```

In [3]:
```
sbux
```

Out[3]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_r
0	Brewed Coffee - Dark Roast	Short	No Milk	No Whip	236.0	3.0	0.1	0.0	0.0	
1	Brewed Coffee - Dark Roast	Tall	No Milk	No Whip	354.0	4.0	0.1	0.0	0.0	
2	Brewed Coffee - Dark Roast	Grande	No Milk	No Whip	473.0	5.0	0.1	0.0	0.0	
3	Brewed Coffee - Dark Roast	Venti®	No Milk	No Whip	591.0	5.0	0.1	0.0	0.0	
4	Brewed Coffee Traveler - Dark Roast	1 - 236 mL serving	No Milk	No Whip	236.0	3.0	0.1	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	
1200	White Hot Chocolate	Venti®	Whole	Whip	591.0	640.0	28.0	18.0	0.5	7
1201	White Hot Chocolate	Venti®	Soy	No Whip	591.0	470.0	14.0	7.0	0.0	
1202	White Hot Chocolate	Venti®	Soy	Whip	591.0	540.0	21.0	11.0	0.2	3
1203	White Hot Chocolate	Venti®	Coconut	No Whip	591.0	450.0	17.0	15.0	0.0	
1204	White Hot Chocolate	Venti®	Coconut	Whip	591.0	520.0	24.0	20.0	0.2	3

1205 rows × 20 columns

In [4]:
```
sbux[sbux.isnull().all(axis=1)]
```

Out[4]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_mg	Sod
11	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

In [5]:
```
sbux.head()
```

Out[5]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_mg	So
0	Brewed Coffee - Dark Roast	Short	No Milk	No Whip	236.0	3.0	0.1	0.0	0.0	0.0	
1	Brewed Coffee - Dark Roast	Tall	No Milk	No Whip	354.0	4.0	0.1	0.0	0.0	0.0	
2	Brewed Coffee - Dark Roast	Grande	No Milk	No Whip	473.0	5.0	0.1	0.0	0.0	0.0	
3	Brewed Coffee - Dark Roast	Venti®	No Milk	No Whip	591.0	5.0	0.1	0.0	0.0	0.0	
4	Brewed Coffee Traveler - Dark Roast	1 - 236 mL serving	No Milk	No Whip	236.0	3.0	0.1	0.0	0.0	0.0	

In [6]:
```
sbux2 = sbux.dropna(how="all")
```

In [7]:
```
sbux2[sbux2.isnull().all(axis=1)]
```

Out[7]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_mg	Sodium

In [8]:
```
sbux2.dtypes
```

Out[8]:
```
Product_Name            object
Size                    object
Milk                    object
Whip                    object
Serving_Size_ml        float64
Calories               float64
Total_Fat_g            float64
Saturated_Fat_g        float64
Trans_Fat_g            float64
Cholesterol_mg         float64
Sodium_mg              float64
Carbohydrates_g        float64
Dietary_Fiber_g        float64
Sugar_g                 object
Protein_g              float64
Vitamin_A_%DV          float64
Vitamin_C_%DV          float64
Calcium_%DV            float64
Iron_%DV               float64
Caffiene_milligrams    float64
dtype: object
```

In [9]:
```
sbux2["Sugar_g"] = sbux2["Sugar_g"].astype(float)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 sbux2["Sugar_g"] = sbux2[          ].astype(float)

File /opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/pandas/core/generic.py:6665, in NDFrame.asty
pe(self, dtype, copy, errors)
   6659     results = [
   6660         ser.astype(dtype, copy=copy, errors=errors) for _, ser in self.items()
   6661     ]
   6663 else:
   6664     # else, only a single dtype is given
-> 6665     new_data = self._mgr.astype(dtype=dtype, copy=copy, errors=errors)
   6666     res = self._constructor_from_mgr(new_data, axes=new_data.axes)
   6667     return res.__finalize__(self, method="astype")

File /opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/pandas/core/internals/managers.py:449, in Ba
seBlockManager.astype(self, dtype, copy, errors)
    446 elif using_copy_on_write():
    447     copy = False
--> 449 return self.apply(
    450             ,
    451     dtype=dtype,
    452     copy=copy,
    453     errors=errors,
    454     using_cow=using_copy_on_write(),
    455 )

File /opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/pandas/core/internals/managers.py:363, in Ba
seBlockManager.apply(self, f, align_keys, **kwargs)
    361         applied = b.apply(f, **kwargs)
    362     else:
--> 363         applied = getattr(b, f)(**kwargs)
    364     result_blocks = extend_blocks(applied, result_blocks)
    366 out = type(self).from_blocks(result_blocks, self.axes)

File /opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/pandas/core/internals/blocks.py:784, in Bloc
k.astype(self, dtype, copy, errors, using_cow, squeeze)
    781         raise ValueError("Can not squeeze with more than one column.")
    782     values = values[0, :]  # type: ignore[call-overload]
--> 784 new_values = astype_array_safe(values, dtype, copy=copy, errors=errors)
    786 new_values = maybe_coerce_values(new_values)
```

```
    788 refs = None

File /opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/pandas/core/dtypes/astype.py:237, in astype_
array_safe(values, dtype, copy, errors)
    234     dtype = dtype.numpy_dtype
    236 try:
--> 237     new_values = astype_array(values, dtype, copy=copy)
    238 except (ValueError, TypeError):
    239     # e.g. _astype_nansafe can fail on object-dtype of strings
    240     #  trying to convert to float
    241     if errors == "ignore":

File /opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/pandas/core/dtypes/astype.py:182, in astype_
array(values, dtype, copy)
    179     values = values.astype(dtype, copy=copy)
    181 else:
--> 182     values = _astype_nansafe(values, dtype, copy=copy)
    184 # in pandas we don't store numpy str dtypes, so convert to object
    185 if isinstance(dtype, np.dtype) and issubclass(values.dtype.type, str):

File /opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/pandas/core/dtypes/astype.py:133, in _astype
_nansafe(arr, dtype, copy, skipna)
    129     raise ValueError(msg)
    131 if copy or arr.dtype == object or dtype == object:
    132     # Explicit copy, or required since NumPy can't view from / to object.
--> 133     return arr.astype(dtype, copy=True)
    135 return arr.astype(dtype, copy=copy)

ValueError: could not convert string to float: 'zero'
```

In [10]:
```
sbux2[sbux2["Sugar_g"] == "zero"]
```

Out[10]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_mg	Sodiu
1	Brewed Coffee - Dark Roast	Tall	No Milk	No Whip	354.0	4.0	0.1	0.0	0.0	0.0	

In [11]:
```
sbux2.loc[1, "Sugar_g"] = 0
```

In [12]:
```
sbux2.loc[1, "Sugar_g"]
```

Out[12]:
```
0
```

In [13]:
```
sbux2["Sugar_g"] = sbux2["Sugar_g"].astype(float)
```

```
/tmp/ipykernel_381/3190073667.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning
-a-view-versus-a-copy
  sbux2["Sugar_g"] = sbux2["Sugar_g"].astype(float)
```

In [14]:
```
sbux2["Sugar_g"].dtype
```

Out[14]:
```
dtype('float64')
```

In [15]:
```
sbux2[sbux2.isnull().any(axis=1)]
```

Out[15]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_mg	S
15	Brewed Coffee Traveler - Medium Roast	1 - 236 mL serving	No Milk	No Whip	236.0	3.0	0.1	NaN	0.0	0.0	

In [16]:
```
sbux3 = sbux2.dropna()
```

In [17]:
```
sbux3[sbux3.isnull().any(axis=1)]
```

Out[17]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_mg	Sodiu

In [18]:
```
sbux3.columns
```

Out[18]:
```
Index(['Product_Name', 'Size', 'Milk', 'Whip', 'Serving_Size_ml', 'Calories',
       'Total_Fat_g', 'Saturated_Fat_g', 'Trans_Fat_g', 'Cholesterol_mg',
       'Sodium_mg', 'Carbohydrates_g', 'Dietary_Fiber_g', 'Sugar_g',
       'Protein_g', 'Vitamin_A_%DV', 'Vitamin_C_%DV', 'Calcium_%DV',
       'Iron_%DV', 'Caffiene_milligrams'],
      dtype='object')
```

In [19]:
```
sbux3 = sbux3.rename(columns={"Caffiene_milligrams": "Caffeine_milligrams"})
```

In [20]:
```
sbux3.columns
```

Out[20]:
```
Index(['Product_Name', 'Size', 'Milk', 'Whip', 'Serving_Size_ml', 'Calories',
       'Total_Fat_g', 'Saturated_Fat_g', 'Trans_Fat_g', 'Cholesterol_mg',
       'Sodium_mg', 'Carbohydrates_g', 'Dietary_Fiber_g', 'Sugar_g',
       'Protein_g', 'Vitamin_A_%DV', 'Vitamin_C_%DV', 'Calcium_%DV',
       'Iron_%DV', 'Caffeine_milligrams'],
      dtype='object')
```

In [21]:
```
sbux3.to_csv("Starbucks_Beverage_Menu_Cleaned.csv", index=False)
```

In [22]:
```
sbux_clean = pd.read_csv("Starbucks_Beverage_Menu_Cleaned.csv")
```

In [23]:
```
sbux_clean.head()
```

Out[23]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Saturated_Fat_g	Trans_Fat_g	Cholesterol_mg	So
0	Brewed Coffee - Dark Roast	Short	No Milk	No Whip	236.0	3.0	0.1	0.0	0.0	0.0	
1	Brewed Coffee - Dark Roast	Tall	No Milk	No Whip	354.0	4.0	0.1	0.0	0.0	0.0	
2	Brewed Coffee - Dark Roast	Grande	No Milk	No Whip	473.0	5.0	0.1	0.0	0.0	0.0	
3	Brewed Coffee - Dark Roast	Venti®	No Milk	No Whip	591.0	5.0	0.1	0.0	0.0	0.0	
4	Brewed Coffee Traveler - Dark Roast	1 - 236 mL serving	No Milk	No Whip	236.0	3.0	0.1	0.0	0.0	0.0	

In [24]:
```
sbux_clean.shape == sbux3.shape
```

Out[24]:
```
True
```

In [ ]: